

Descrição geral do trabalho:

O objetivo do trabalho é implementar uma árvore binária de busca com persistência parcial seguindo o método descrito em sala. A entrada do programa é um arquivo de texto (.txt), que deve ser recebido como argumento do programa, com várias operações sobre a estrutura (uma por linha) e a saída será o resultado das operações impressos no próprio terminal que estiver sendo utilizado (apenas daquelas cujo resultado pode ser impresso).

Variação para pós-graduação:

Para a pós-graduação apenas, a árvore binária de busca deve ser uma árvore rubro-negra.

Operações:

- **Inclusão:** Uma operação de inclusão será identificada por uma linha escrito INC seguido de um espaço e depois um inteiro. Este elemento deve ser incluído na estrutura de dados, uma nova versão criada e nada precisa ser impresso.
Exemplo de linha de inclusão:
INC 13
- **Remoção:** Uma operação de remoção será identificada por uma linha escrito REM seguido de um espaço e depois um inteiro. Um nó com este valor deve ser removido (apenas um se houver repetição). Caso não haja um nó com o valor especificado, a estrutura não deve ser alterada. Em ambos os casos uma nova versão deve ser criada e nada precisa ser impresso.
Exemplo de linha de remoção:
REM 42
- **Sucessor:** Uma operação de sucessor será identificada por uma linha escrito SUC seguido de um espaço, um inteiro, outro espaço e outro inteiro. O sucessor do número x é o menor valor na estrutura que é estritamente maior que x. O primeiro inteiro será o valor (que não precisa estar na estrutura) cujo sucessor deseja-se saber e o segundo será a versão em que o sucessor deverá ser procurado. Caso a versão fornecida não exista, o sucessor deve ser procurado na versão mais recente e caso não exista valor na estrutura maior que o primeiro inteiro fornecido, o sucessor será infinito. Essa operação não cria uma nova versão na estrutura e imprime a própria operação e o resultado no terminal.
Exemplo de linha de sucessor:
SUC 50 65
Exemplo dessa linha na impressão de saída:
SUC 50 65
52
- **Imprimir:** Uma operação de impressão será identificada por uma linha escrito IMP seguido de um espaço e um inteiro. O inteiro indica a versão da estrutura que deve ser impressa, isto é, cada elemento da estrutura na versão indicada deve ser impresso em ordem crescente seguido da sua profundidade (e cor para a pós-graduação) separados por vírgula. Caso a versão fornecida não exista, a impressão deve ocorrer na versão mais recente. Essa operação não cria uma nova versão na estrutura e imprime a própria operação e o

resultado no terminal.

Exemplo de linha de impressão:

IMP 65

Exemplo dessa linha na impressão de saída:

IMP 65

13,2 42,1 50,0 52,2 65,1 (Versão da Graduação sem cores)

13,2,R 42,1,N 50,0,N 52,2,R 65,1,N (Versão da Pós-Graduação com cores)

Versões:

A estrutura inicialmente começa na versão 0. Cada operação de inclusão e remoção aumenta a versão da estrutura em 1. Haverá no máximo 99 operações de inclusão e remoção, de modo que haverá no máximo 100 versões diferentes da estrutura, então os identificadores das versões (raiz da estrutura e quais em quais versões ela opera) podem ser guardados num vetor de tamanho 100. Não há limite para o número de operações de sucessor e de impressão, mas estas não criam novas versões.

Grupos e Entrega:

- Cada equipe da graduação pode ser formada por até 3 membros.
- Cada equipe da pós-graduação pode ser formada por até 2 membros.

O envio do trabalho deve conter:

- O seu código-fonte
- Um arquivo **makefile** que possua funcionalidades para compilar (**build**) e executar (**run**) o seu programa.
 - Lembre-se que seu programa deve receber um arquivo de texto (.txt) como entrada.
 - Logo, o comando para a funcionalidade **run** deve ser capaz de receber um argumento que é o endereço de um arquivo de texto que será a entrada do programa.
- Um arquivo **README.md** de descrição do trabalho que indica:
 - A linguagem de programação usada (incluindo a versão usada)
 - Como rodar o seu programa
 - Descrição das funções e estruturas (equivalente ao struct em C) que são usadas no seu código
 - Em quais arquivos cada uma dessas funções e estruturas se encontram.

Exemplo do makefile (com todos os arquivos no mesmo diretório):

```
makefile
1  INPUT = entrada.txt
2
3  build: main.cpp
4      |   g++ -o programa.exe main.cpp
5
6  run: programa.exe
7      |   ./programa.exe $(INPUT)
8
```

Nesse exemplo, temos 2 comandos para o make: *build* e *run*

O comando **make build** irá executar a linha 4 (abaixo da definição do *build*). Ele precisa que o arquivo **main.cpp** exista no diretório (por isso está sendo especificado depois dos dois pontos na linha 3). Ele irá compilar o conteúdo do arquivo no executável **programa.exe**.

Já o comando **make run** irá executar a linha 7. Ele precisa que o arquivo **programa.exe** exista no diretório (ou seja, só pode ser executado depois do *build*). Ele irá executar o programa, passando como parâmetro o endereço de um arquivo de texto (assumindo que o código feito execute recebendo um arquivo .txt como argumento de entrada).

O **INPUT** é uma variável (de string) definida dentro do arquivo do *makefile*. Por padrão definimos ele como **entrada.txt**. Se executarmos apenas o comando *make run*, ele irá executar a linha 7 substituindo o **\$(INPUT)** pela string salva nele, e com isso, rodará o *programa.exe* passando o arquivo *entrada.txt* como entrada (se ele existir).

Se quisermos usar um arquivo com nome diferente do padrão, basta executar o comando **make run INPUT="nome do arquivo"**. Por exemplo, com um arquivo de nome **entrada2.txt**:

```
make run INPUT=entrada2.txt
```

O que estamos fazendo é atribuir outra *string* à variável, antes de rodar o comando. Assim, o comando irá rodar o **programa.exe** passando como argumento um arquivo com o nome (*entrada2.txt*) que colocamos no comando.

Testes:

Sugestão para realização de testes (com todos os arquivos no mesmo diretório):

Para cada arquivo de entrada (**entrada.txt** por exemplo), mantenha um arquivo com a saída esperada para essa entrada (**saida_esperada.txt** por exemplo).

Então, após compilar seu programa (**programa.exe** por exemplo), rode o seguinte comando no terminal (no diretório onde os arquivos estão):

```
./programa.exe entrada.txt > saida.txt
```

Isso irá copiar todas as linhas que foram impressas durante a execução do programa usando o arquivo *entrada.txt* como entrada, e irá salvá-las em um novo arquivo **saida.txt** no mesmo diretório.

Após isso, rode o seguinte comando no terminal (no diretório onde os arquivos estão):

```
diff saida.txt saida_esperada.txt > diferencas.txt
```

(No terminal cmd do Windows, use o comando **fc** em vez de **diff**)

Isso irá comparar os 2 arquivos e gerar um arquivo **diferencas.txt** no mesmo diretório, que irá conter todas as diferenças entre a saída obtida e a saída esperada para a entrada usada.

Exemplo:

Seja o arquivo **entrada.txt** que possui as seguintes linhas de operações:

```
INC 1
INC 2
IMP 1
IMP 2
```

E seja o arquivo **saida_esperada.txt**, que possui as seguintes linhas esperadas para a execução do arquivo **entrada.txt**:

```
IMP 1
1,0,N
IMP 2
1,0,N 2,1,R
```

Durante a execução do programa foi passado o arquivo **entrada.txt** como entrada, e, usando o comando mostrado, ele gerou o arquivo **saida.txt** com as seguintes linhas:

```
IMP 1
1,0,N
IMP 2
1,0,R 2,1,N
```

Ao executar o comando *diff* mostrado, para comparar a saída obtida com a saída esperada, isso irá gerar um arquivo **diferencas.txt** com as seguintes linhas:

```
4c4
< 1,0,R 2,1,N
---
> 1,0,N 2,1,R
```

(Se usar o comando **fc** do Windows, o arquivo será um pouco diferente, mas a mesma ideia ainda se aplica)

Essas linhas significam que houve diferença na linha 4 entre os dois arquivos.

Isso indica que provavelmente existe um erro durante a segunda operação INC, ou na própria operação de impressão IMP, realizadas durante a execução do programa.